



Mestrado em Eng. Informática

2025/26

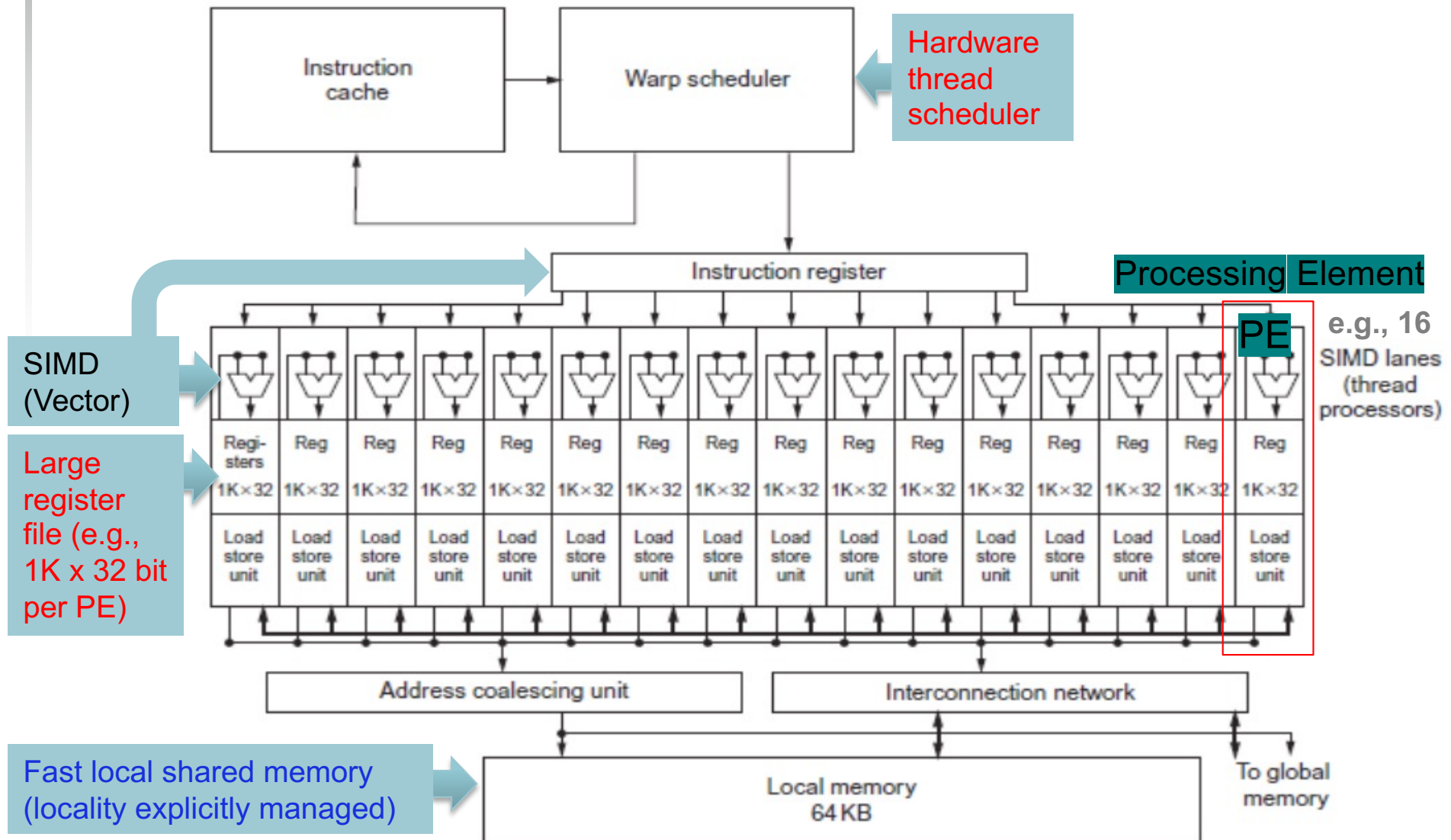
J.L. Sobral

Computing accelerators: GPUs

Part 2 : thread blocks & shared memory

Recap: GPU Organization (NVIDIA)

A Multithreaded **SIMD** Processor (NVIDIA: SM - Streaming Multiprocessor)



GK110 Kepler Architecture (K20 GPU on our cluster)

Kepler:15 SM, 2880 PEs *October'13*
Each SM is like a PU on a multicore

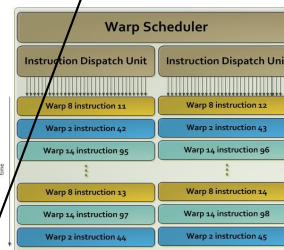
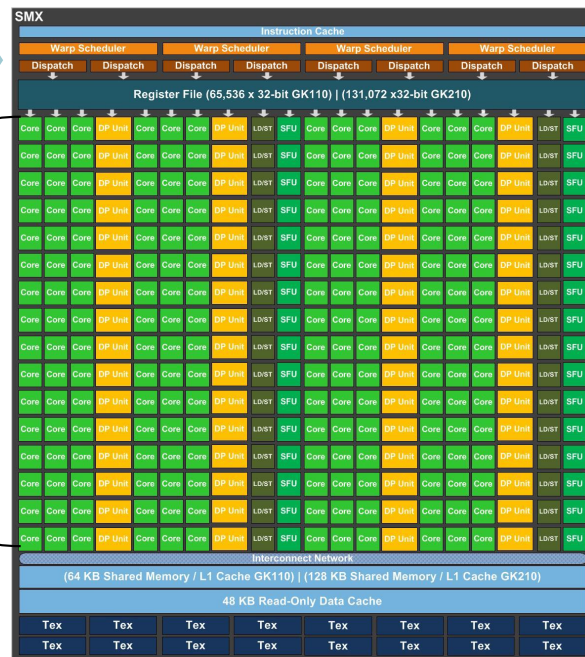
SMs internally rely on vector processing

Each SM contains (GK110):

- 192 PEs FP32 (green PEs)
- 64 PEs FP64 (yellow PEs)
- 32 Load/store units & 32 SFU



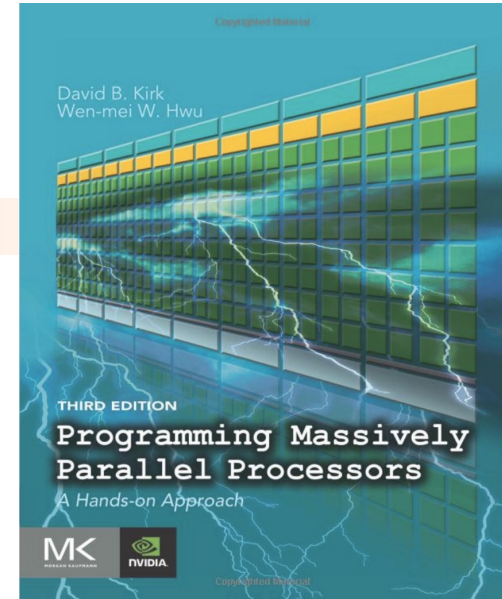
SIMD lanes



Quad Warp Scheduler

The SM **schedules** threads in **groups of 32 parallel threads** called warps. Each SM features four warp schedulers and eight instruction dispatch units, allowing four warps to be issued and executed concurrently. **Kepler's quad warp scheduler selects four warps, and two independent SIMD instructions per warp can be dispatched each cycle.**

Recap: CUDA programming model



- **Compute Unified Device Architecture**
- CUDA is a programming model for
 - a multicore **CPU host** coupled to a many-core **device**, where
 - *devices* have wide SIMD/SIMT parallelism, and
 - the *host* and the *device* do not share memory
- CUDA provides:
 - a thread abstraction to deal with SIMD
 - synchr. & data sharing between small groups of threads
- CUDA programs are written in C with extensions
- OpenCL inspired by CUDA, but hw & sw vendor neutral
 - programming model essentially identical

Recap: CUDA Devices and Threads

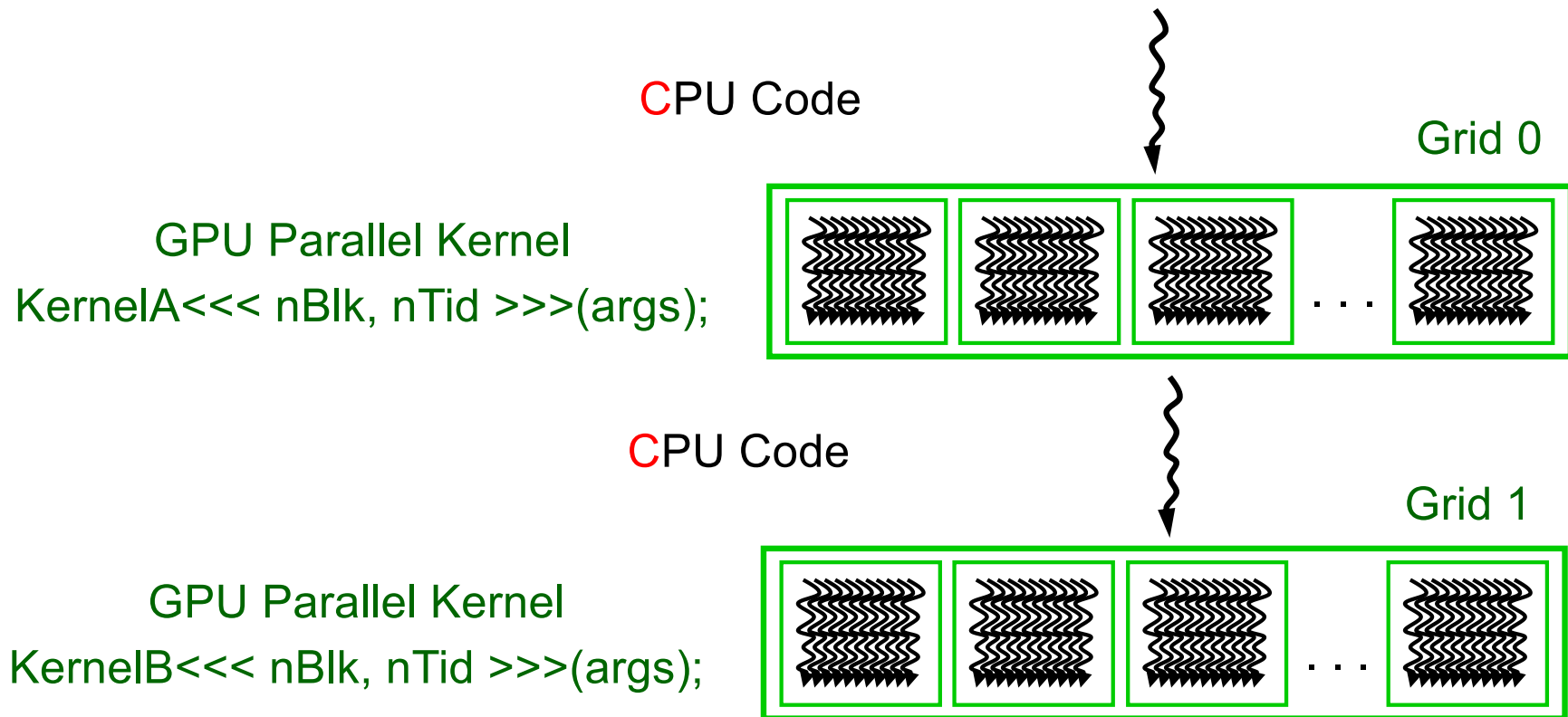


- A compute device
 - is a coprocessor to the CPU or host
 - has its own DRAM (device memory)
 - runs many threads in parallel
 - is typically a GPU but can also be another type of parallel processing device
- Data-parallel portions of an application are expressed as device kernels which run on many threads - **SIMT**
- Differences between GPU and CPU threads
 - GPU threads are extremely lightweight
 - very little overhead, requires LARGE register file
 - GPU needs 1000s of threads for full efficiency
 - multi-core CPU needs only a few

Recap: CUDA basic model: Single-Program Multiple-Data (SPMD)



- CUDA integrated **C**PU + GPU application C program
 - Serial C code executes on **C**PU
 - Parallel **K**ernel C code executes on GPU **thread blocks**



Recap: Code example



C with CUDA Extensions: C with a few keywords

```
void saxpy_serial(int n, float a, float *x, float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}
// Invoke serial SAXPY kernel
saxpy_serial(n, 2.0, x, y);
```

Standard C Code

```
__global__ void saxpy_parallel(int n, float a, float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
// Invoke parallel SAXPY kernel with 256 threads/block
int nblocks = (n + 255) / 256;
saxpy_parallel<<<nblocks, 256>>>(n, 2.0, x, y);
```

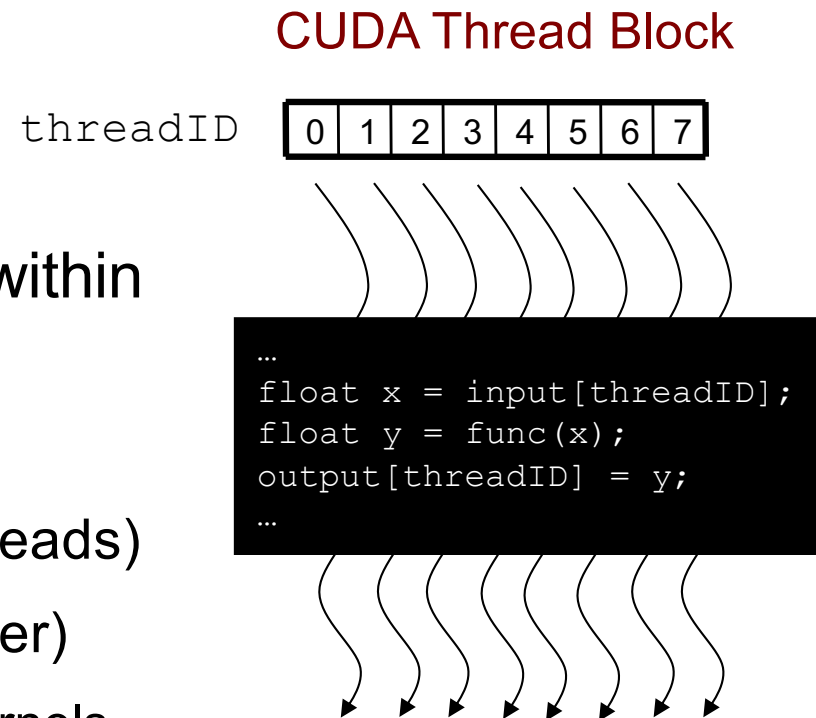
Parallel C Code

NVIDIA Confidential

CUDA Thread Block



- Programmer declares (Thread) Block:
 - Block size: **1** to **1024** threads
 - Block shape: 1D, 2D, or 3D
 - Block dimensions in threads
- Threads have **thread id** numbers within a Block (**id.x**, **id.y**, etc ...)
- **Why thread blocks matter?**
 - Wrap-based SM schedule (32 threads)
 - faster synchronization (local barrier)
 - Global barriers across multiple kernels
 - Local data share and exchange
 - A thread block can shared data on the SM **local shared memory**

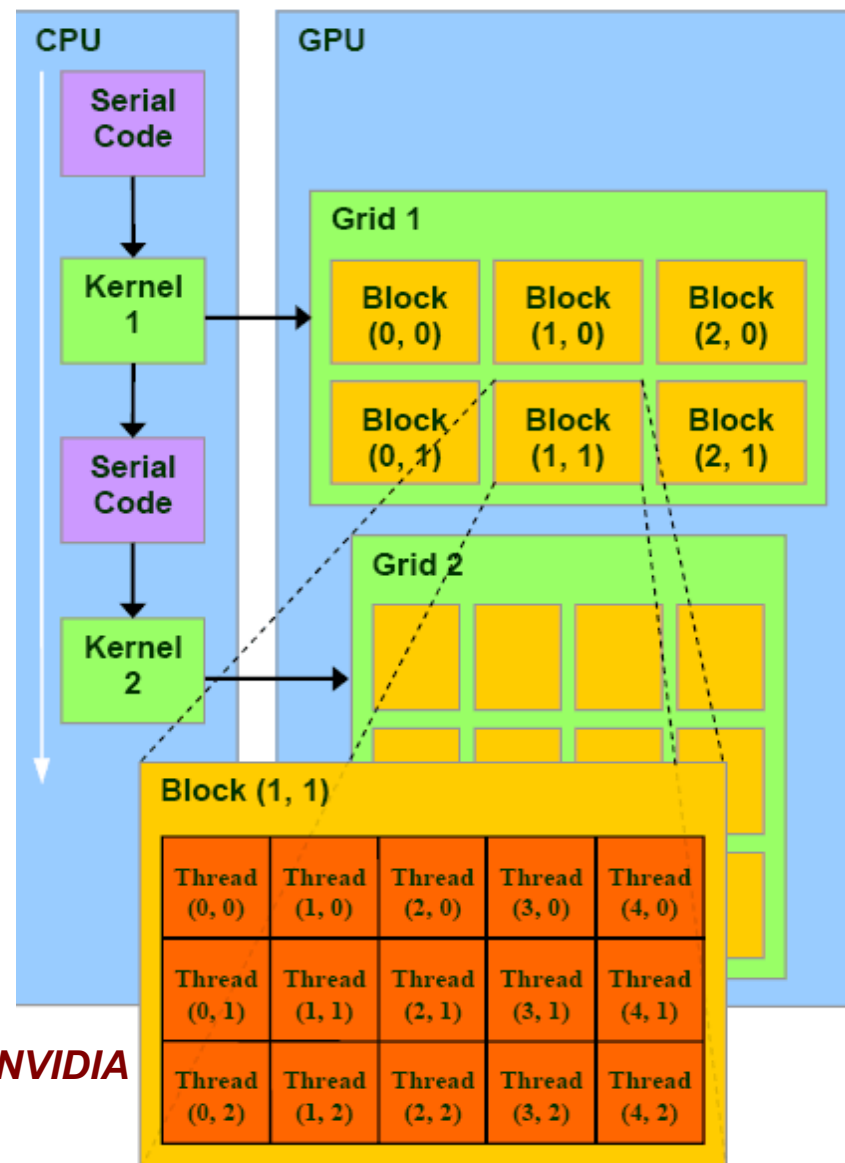
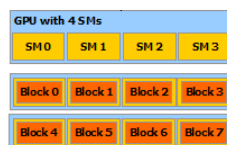
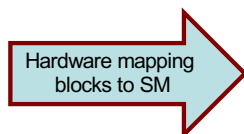
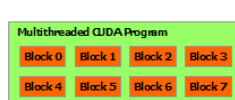


CUDA Thread Hierarchy

Model: SPMD + SIMT/SIMD



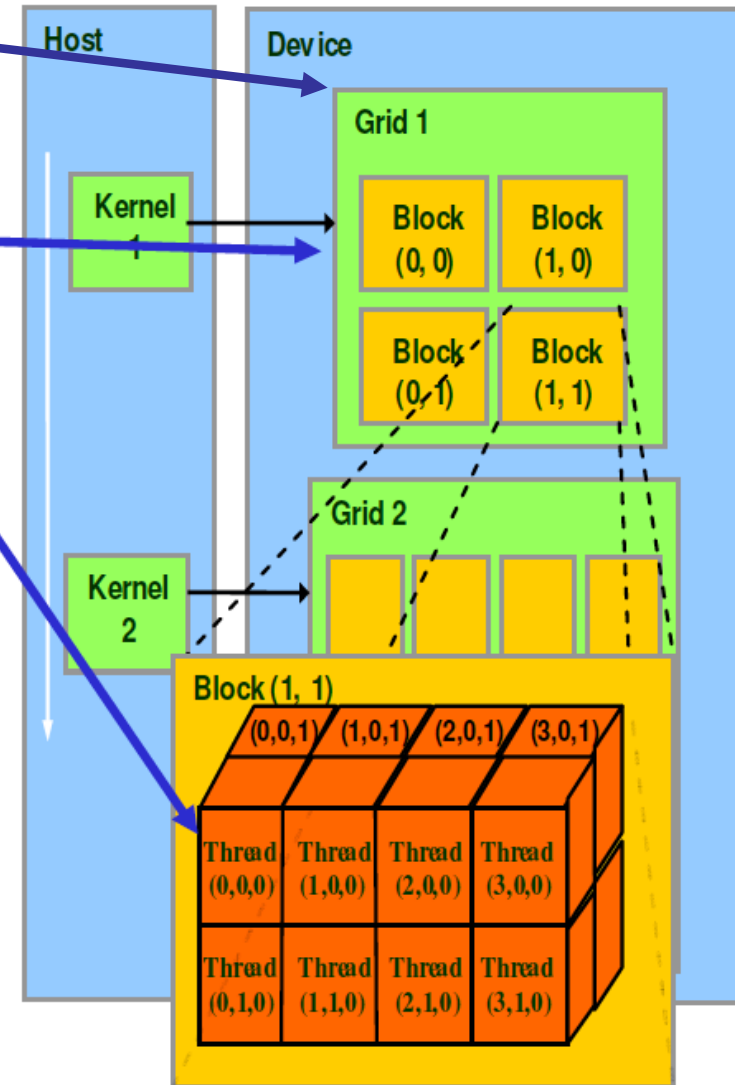
- Thread hierarchical Model
 - Thread** – thread unique id
 - Thread **Block** (max 3 dimensions)
 - Can cooperate (fast sync. & data share)
 - Execute the same code (on a single SM)
 - Grid** (max 2 dimensions)
 - blocks are organised into grids
 - No inter-block cooperation
- Single kernel runs on multiple blocks (SPMD)
- Threads within a warp are executed in a lock-step way called single-instruction multiple-thread (SIMT)
- Single instruction are executed on multiple threads (SIMD)



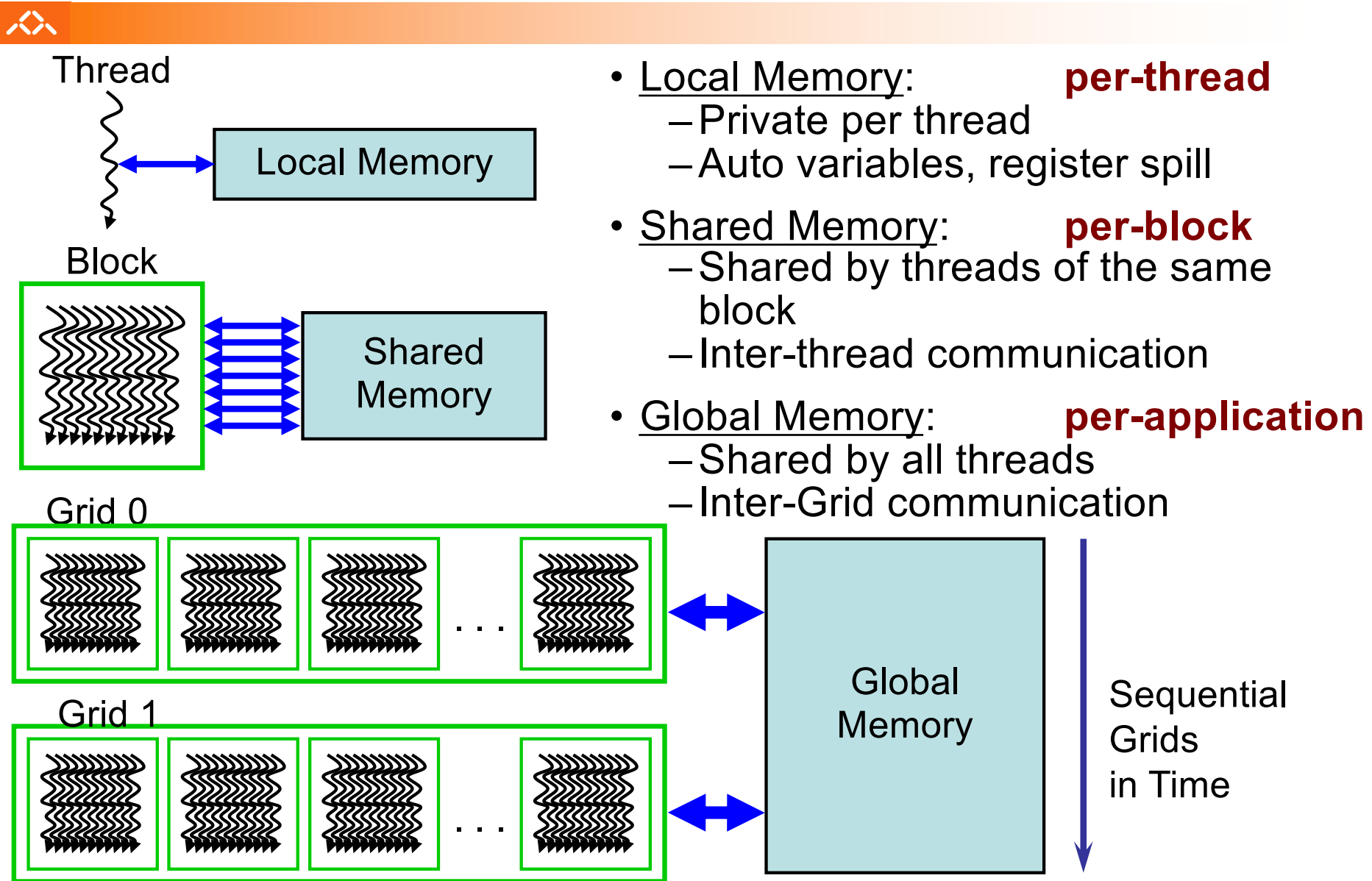
Courtesy NVIDIA

The Computational Grid: Block IDs and Thread IDs

- A **kernel** runs on a **computational grid of thread blocks**
 - Threads share global memory
- Each thread uses IDs to decide what data to work on
 - Block ID: 1D or 2D
 - Thread ID: 1D, 2D, or 3D
- A thread block is a batch of threads that can cooperate by:
 - Sync their execution w/ barrier
 - Efficiently sharing data through a low latency shared memory
 - Two threads from two different blocks cannot cooperate



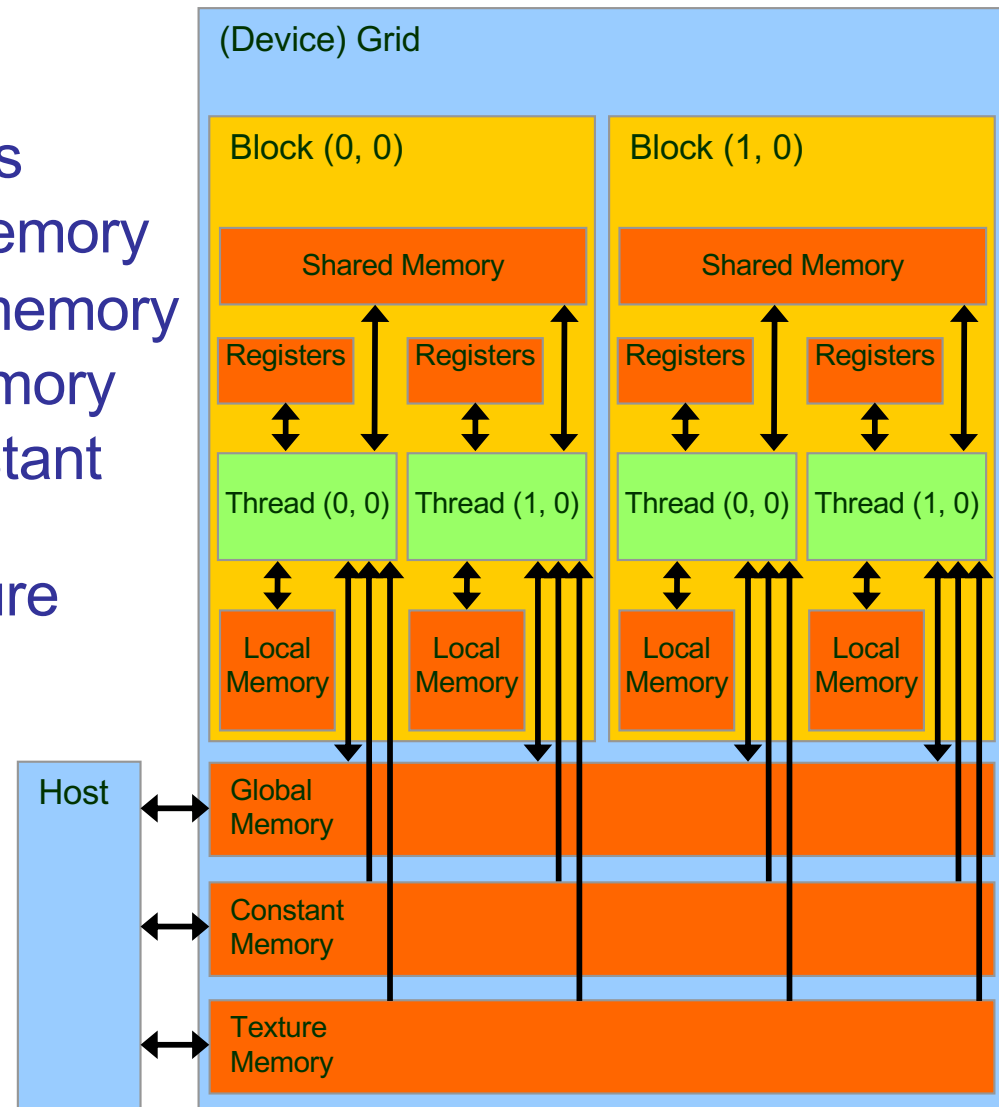
Parallel Memory Sharing



CUDA Memory Model Overview



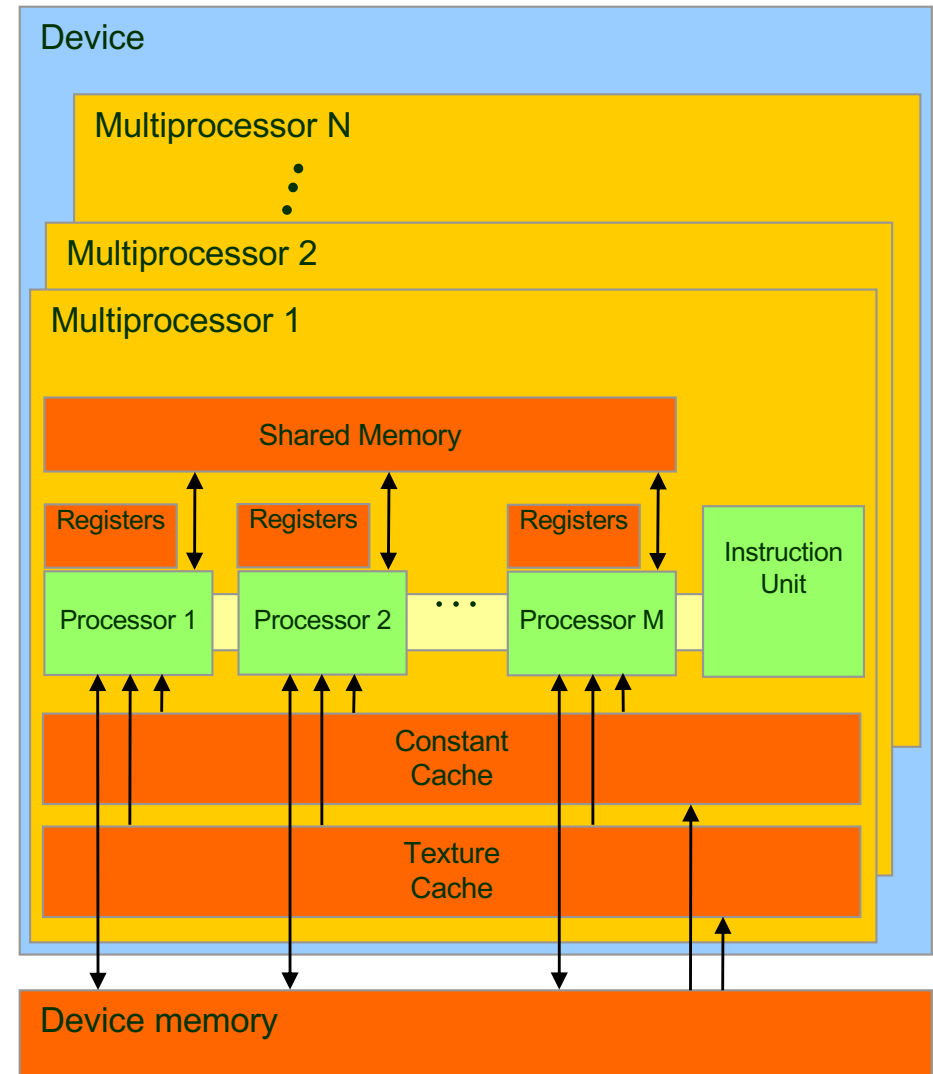
- Each thread can:
 - R/W per-thread registers
 - R/W per-thread local memory
 - R/W per-block shared memory
 - R/W per-grid global memory
 - Read only per-grid constant memory
 - Read only per-grid texture memory
- The host can R/W global, constant, and texture memories



Hardware Implementation: Memory Architecture



- Device memory (DRAM)
 - Slow (2~300 cycles)
 - Local, global, constant, and texture memory
- On-chip memory
 - Fast (1 cycle)
 - Registers, shared memory, constant/texture cache



Courtesy NVIDIA

Recap: Vector Programming (& challenges)

- Compilers are a key element to give hints on whether a code section will vectorize or not

- Check if loop iterations have **data dependencies** and/or if...then statements, otherwise vectorization is compromised

- Vector architectures have a too high cost, but simpler variants are currently available on off-the-shelf devices, as SIMD extensions to the scalar PUs; **however**:

- most do not support **non-unit stride**
=> care must be taken in the design of data structures (data layout)
- same applies for **mask register**,
- **gather-scatter...**



No data dependencies allowed

- Vector prog. applies only to data-parallelism (e.g. map pattern)

Deal with control dependencies

- common solutions:
 - predication => additional work
 - mask registers => unused PEs

Support for arbitrary vector lengths

- when the vector registers size does not match the vector length

Maximize data locality

- maximize spatial locality (next slides)
- data alignment in memory

NOTE: SIMD consumes much more data per instruction!

CUDA optimization example: Reduction



- **Recap:** Common **data parallelism** patterns
 - **Map:** applies side-effect free unary function to all input elements

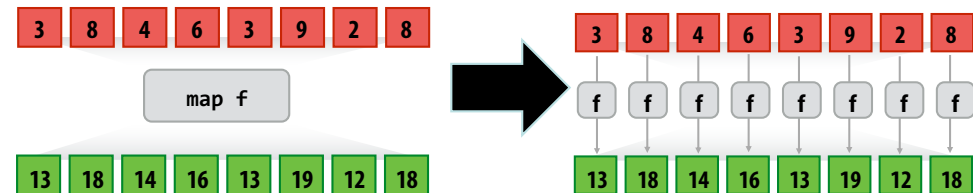
C++

```
int f(int x) { return x + 10; }
```

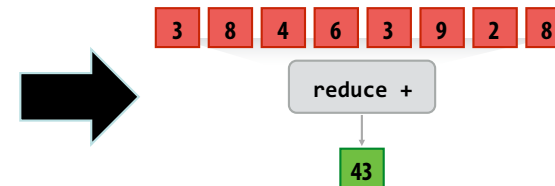
```
int a[] = {3, 8, 4, 6, 3, 9, 2, 8};
```

```
int b[8];
```

```
std::transform(a, a+8, b, f);
```



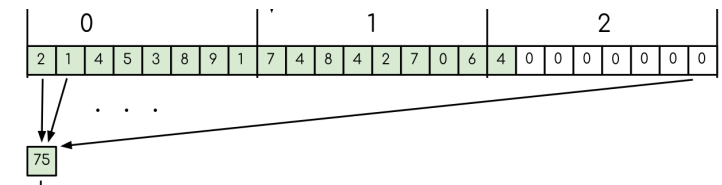
- **Reduce:** Applies a binary operation returning an accumulated value



- **Naive GPU reduce implementation** (for $\#threads < \#itens$)

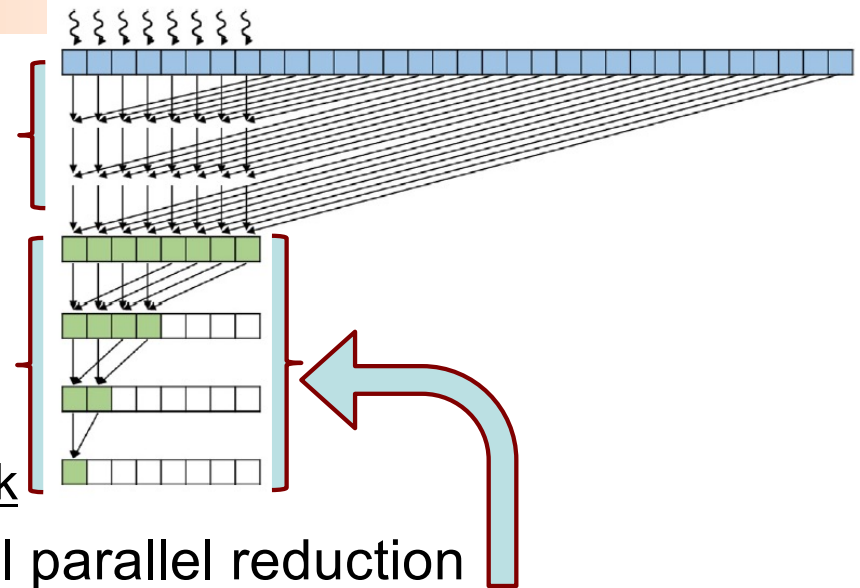
Local “sequential” reduce of $\#itens/\#threads$ itens
& atomic operation on global memory (atomicAdd)

- Main problems:
 - no spatial locality when reading global memory
 - sequential access on final write to global memory (one atomic operation per thread!)



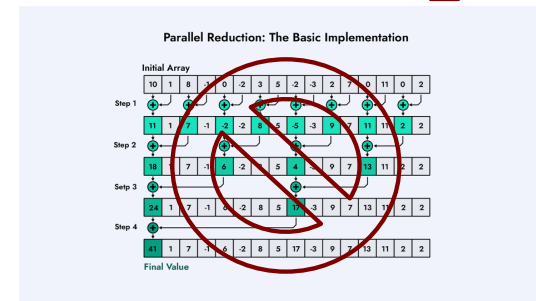
CUDA optimization example: Reduction

- Three **illustrative** improvements
 - Improve global **memory accesses**
 - Spatial locality** (a.k.a. coalescing)
 - Local **parallel reduction** on **ShMem**
 - Store intermediate results on **ShMem**
 - One atomic operation per thread block
 - Minimize **control divergence** on local parallel reduction

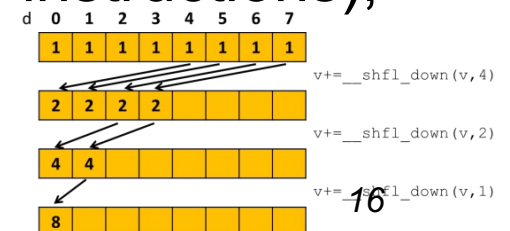
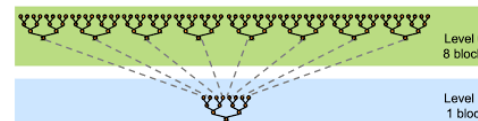


```

for (unsigned int stride = blockDim.x; stride >= 1; stride /= 2) {
    for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
        if (threadIdx.x < stride) if (threadIdx.x % stride == 0)
            sm_input[i] += sm_input[i + stride];
        __syncthreads(); // local thread block barrier
    }
}
    
```



- Other:** reduce **intra-block synchronisation** (shuffle instructions),
multiple reduce **trees**



Parallel Computing Course – next weeks



- **Today** - WA phase 1 – problem understanding & parallelism - OpenMP
- **Next week** - WA phase 1 – feed-back
 - final WA - same code, different simulation; more freedom (MPI; CUDA;)
- **Next two weeks (1 to 12/Dec)**
 - Holliday on Mondays (1 & 8) => **no PL lectures**
- **Week 15-17 December** - No T lecture; project support on PL lectures
- **9/Jan/2026** - Written test
- **Week from 12 to 16 January 2025**
 - Final WA submission (**Monday-12/jan**) & defence (14 & 15)